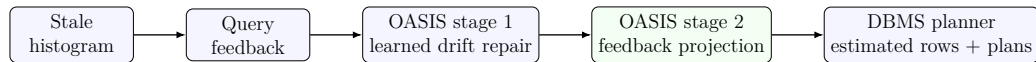


Graphical Abstract

OASIS: Feedback-Driven Marginal Statistics Correction for Cost-Based Query Optimizers

Author Names Redacted for Review



Highlights

OASIS: Feedback-Driven Marginal Statistics Correction for Cost-Based Query Optimizers

Author Names Redacted for Review

- OASIS pairs a learned single-column drift-repair stage with a feedback-consistency projection.
- Both stages are necessary: learning supplies a reusable drift prior; projection makes the marginal safe to compose.
- Stage-2 calibration is regime-dependent; a residual-gated router adds a consistency-gated soft projection and improves single-column and optimizer-facing accuracy with no join/planner regression.
- OASIS supplies drop-in marginals for copula, IPF, and FactorJoin join estimators.
- Deployment safety is evaluated with traces, feedback noise, and PostgreSQL planner checks.

OASIS: Feedback-Driven Marginal Statistics Correction for Cost-Based Query Optimizers

Author Names Redacted for Review^a

^a*Affiliation Redacted, , ,*

Abstract

Cost-based optimizers depend on column-level statistics that are refreshed by periodic **ANALYZE** passes. Between refreshes, continuous inserts, deletes, and updates create a staleness gap: the optimizer keeps planning with obsolete histograms even though query execution exposes predicate-level feedback. This paper presents OASIS, a two-stage statistics-layer correction pipeline: a learned drift-repair model first predicts corrected single-column marginals from stale histograms and recent query feedback, and a feedback-consistency projection then calibrates the repaired marginal before it is exposed to downstream estimators or a DBMS planner. OASIS maps engine-native statistics to a canonical quantile/CDF view, requires no base-table rescans, and leaves the optimizer’s search procedure and cost model unchanged.

The evaluation is optimizer-facing rather than runtime-facing, and it shows that both stages are necessary. The learned repair stage alone reduces single-column selectivity Q-error by 51.3%, 57.9%, and 60.6% at drift intensities $q = 10, 20$, and 30 , outperforming local-feedback methods such as ISOMER in the moderate-to-large drift regime; such methods are constrained by observed feedback and do not learn a reusable global drift-repair prior. The projection stage is what makes the repaired marginal safe to compose: the stage-one marginal alone gives only limited and unstable multi-column gains and is *actively harmful* in the bilinear FactorJoin join kernel, whereas the full two-stage OASIS recovers stable gains. Embedded as a drop-in marginal source across a six-estimator composition family (independence, IPF/Sinkhorn, and four copulas), OASIS improves every estimator in that family by 22%–38%. In the FactorJoin join kernel, OASIS-noProj is harmful, while the full OASIS recovers stable join-cardinality gains. Across 12 PostgreSQL planner-only configurations, OASIS reduces aggregate row-estimation Q-error from 27.748 to 2.360 and matches the fresh-statistics plan

shape on 96.1% of queries, while the projection reduces new plan deviations from 13.8% to 1.1%.

We then show that the Stage-2 calibration is *regime-dependent*: exact (hard) projection is best for join and planner safety, whereas a consistency-gated *soft* projection—which down-weights feedback contradicted by the most recent observations—generalizes better for future single-column and optimizer-facing accuracy. A residual-gated router that adds soft projection to its candidate pool selects per case from a deployment-visible signal: in a held-out future-predicate evaluation it lowers single-column Q-error from 1.442 (hard projection) to 1.395 and optimizer-proxy selectivity Q-error from 1.446 to 1.396, while matching hard projection on the FactorJoin kernel and remaining identical to the deployed Hybrid on the PostgreSQL plan-shape batch, because the gate never routes to soft where exactness matters. A simpler residual-gated Hybrid already supports safe deployment under sparse or noisy feedback, cutting aggregate DML-trace Q-error from 1.295 to 1.072. No query runtime is measured.

Keywords: Statistics correction, Cardinality estimation, Histogram calibration, Cost-based optimizer, Query feedback, Online learning

1. Introduction

Cost-based optimizers (CBOs) choose query plans from estimates of predicate selectivity, intermediate cardinality, and operator cost. In many analytical database systems, these estimates rely on column-level statistics such as histograms, most-common-value lists, and density summaries. The statistics are periodically refreshed through **ANALYZE**-style procedures that read a full or sampled view of the table. This maintenance strategy is simple and robust, but it leaves a persistent staleness gap: tables continue to receive inserts, deletes, and updates while the optimizer keeps planning with the last statistics snapshot. As the gap widens, stale histograms induce systematic selectivity errors that can propagate through the optimizer’s cost model.

Existing remedies operate at different layers of the optimization stack. Frequent **ANALYZE** reduces staleness but increases scan overhead. Learned cardinality estimators predict point cardinalities for particular queries and often require raw data, query workloads, or table-specific training. Plan-level feedback systems adjust plan choices or hints after observing execution behavior, but they do not repair the underlying statistics objects consumed

by the optimizer. This paper focuses on a complementary layer: *statistics-level correction*. Rather than replacing the optimizer or learning complete query cardinalities, we update the column statistics supplied to the existing CBO.

The key observation is that a DBMS already obtains useful feedback while executing ordinary queries. For a filter predicate, the optimizer’s estimated selectivity and the executor’s observed selectivity expose a local discrepancy between the stale histogram and the current data distribution. OASIS turns a recent window of such observations into a corrected single-column histogram before the statistics reach the optimizer. The design is intentionally non-invasive: OASIS requires a query-completion feedback listener and a statistics-assembly hook, but no changes to the optimizer’s search algorithm or cost model.

The central challenge is that feedback is sparse, local, and heterogeneous. Naively forcing a histogram to match a few recent predicates can overfit the feedback window; ignoring the feedback leaves the prior stale. OASIS addresses this problem by learning a *drift-correction function*. It encodes the stale quantile histogram together with recent predicate observations and uses a compact pooled MLP to predict corrected quantile boundaries. The same 38K-parameter checkpoint is trained once on simulation-derived drift scenarios and then reused across columns and schemas in our experiments.

Because OASIS operates at the marginal-statistics layer, its role in multi-column settings is conditional rather than automatic. It supplies corrected one-dimensional inputs; it does not infer dependence structure or repair join correlations by itself. This is why OASIS has two stages. The learned drift-repair stage extrapolates the post-drift marginal from the stale prior and recent feedback, which is what improves estimates on future, unseen predicates beyond the observed feedback window. Local-feedback methods are constrained by observed predicates and lack a learned global drift prior. But a globally plausible learned marginal need not be consistent with the predicates that were just observed, and a downstream estimator can amplify that local inconsistency into a much larger joint error. The second stage therefore projects the learned marginal onto the current feedback constraints before it is consumed by a downstream estimator or DBMS planner. Throughout, we write OASIS-noProj for the stage-one-only output and OASIS for the full two-stage system, which is the deployed form. The central empirical finding is that both stages are necessary—neither the learned stage nor a local-feedback projection alone is sufficient across single-column accuracy and downstream

composition.

A second finding concerns the Stage-2 calibration itself. Exactly matching the observed feedback (hard projection) is the right choice before multi-column composition and for the DBMS planner, where any residual one-dimensional error is amplified by joins. But on future single-column predicates a softer calibration generalizes better: it regularizes the learned marginal instead of overfitting the exact feedback intervals. We make this soft calibration safe with a *consistency gate*—it down-weights feedback contradicted by the most recent observations rather than cutting feedback by age—and we show that no single Stage-2 operator dominates across both regimes. The deployed Stage 2 is therefore a residual-gated *calibrated router* that selects, per case and from a deployment-visible feedback-residual signal, among the stale prior, ISOMER, OASIS-noProj, hard projection, and the consistency-gated soft projection. The router improves single-column and optimizer-facing accuracy over hard projection while reducing exactly to hard projection on the join kernel and the PostgreSQL planner, where the gate does not select soft. A simpler residual-gated Hybrid over stale, ISOMER, OASIS-noProj, and OASIS remains available for sparse or noisy feedback.

The evaluation follows this scope. We evaluate the mechanism where it acts: future selectivity estimates, structural histogram error, downstream marginal composition—including a six-estimator composition family and the FactorJoin join kernel that consume OASIS as a drop-in marginal source—and real PostgreSQL planner estimates and plan shapes. The PostgreSQL study is planner-only: true rows are obtained with `COUNT(*)`, and optimizer-facing evidence comes from `EXPLAIN (FORMAT JSON)` under injected statistics states, without measuring query runtime. To reduce dependence on a single drift simulator, we also evaluate out-of-distribution drift families and benchmark-inspired DML traces with explicit insert, update, and delete events.

The contributions are:

- We formulate feedback-driven single-column statistics correction as a DBMS-facing problem—given a stale histogram and a causal window of predicate feedback, produce corrected statistics that improve future selectivity estimates without rescanning the table—and instantiate a PostgreSQL-style adapter that maps MCV+histogram statistics to a canonical quantile/CDF view and back, isolating engine-specific logic from the correction model.
- We design OASIS, a two-stage statistics-layer pipeline: a lightweight learned

model predicts corrected quantile boundaries from prior quantiles and heterogeneous feedback, and a feedback-consistency projection calibrates the learned marginal before it is exposed downstream.

- We show that both stages are necessary and that the projection is part of the method, not optional post-processing. The learned stage extrapolates drift beyond the observed feedback window, while local-feedback methods are constrained by observed predicates and do not learn a reusable global drift-repair function. Its raw output is not feedback-consistent and is amplified by downstream composition; in the bilinear FactorJoin join kernel the stage-one marginal is actively worse than stale, a failure the projection mitigates.
- We show that the Stage-2 calibration is regime-dependent and design a consistency-gated soft projection plus a residual-gated calibrated router that resolves it. The soft projection down-weights feedback contradicted by the most recent observations; the router selects per case from a deployment-visible signal, improving single-column Q-error (1.442 to 1.395) and optimizer-proxy selectivity Q-error (1.446 to 1.396) over hard projection while reducing to hard projection on FactorJoin and the PostgreSQL plan-shape batch. We also report a negative result: a banded (uncertainty-relaxed) projection does not help, isolating the gain to the router rather than to relaxed matching.
- We give an optimizer-facing evaluation that measures the mechanism where it acts, without query runtime: single-column drift, structural and initial-distribution generalization, a six-estimator composition family (independence, IPF/Sinkhorn, and Gaussian/Clayton/Gumbel/Frank copulas) and the FactorJoin join kernel that consume OASIS as a drop-in marginal source, a 12-configuration PostgreSQL planner-only statistics-injection study, and deployment-safety checks under sparse, noisy, and trace-grounded feedback.

2. Background and Problem Formulation

2.1. Column Statistics and the Staleness Gap

We model a column histogram as an equi-depth quantile summary with B buckets. The internal boundaries $v_1, \dots, v_{B-1}$ correspond to fixed quantile levels $p_1, \dots, p_{B-1}$. A range predicate such as $x < c$ is estimated by evaluating the piecewise-linear CDF implied by those boundaries. Although real DBMS

statistics may combine histograms with auxiliary summaries such as most-common-value lists, density vectors, or null fractions, OASIS operates on a canonical one-dimensional CDF view.

Let H_0 denote the stale histogram captured at the last statistics refresh and H^* denote the current post-drift distribution. For a predicate σ , the optimizer derives an estimate $\hat{s}_{H_0}(\sigma)$ while the executor can later reveal an actual selectivity $s^*(\sigma)$. We measure selectivity error using Q-error:

$$\text{QErr}(\hat{s}, s^*) = \max \left(\frac{\hat{s}}{s^*}, \frac{s^*}{\hat{s}} \right). \quad (1)$$

2.2. Statistics Correction

Given a stale histogram H_0 and an observation window $\mathcal{O} = \{o_1, \dots, o_K\}$, where each observation records a predicate type, predicate boundary, estimated selectivity, and actual selectivity, the goal is to produce corrected statistics $\hat{H}$ such that future predicates are estimated more accurately:

$$\mathbb{E}_\sigma [\text{QErr}(\text{CDF}_{\hat{H}}(\sigma), s_\sigma^*)] < \mathbb{E}_\sigma [\text{QErr}(\text{CDF}_{H_0}(\sigma), s_\sigma^*)].$$

The expectation is over future predicates from the workload distribution.

The solution must satisfy three deployment constraints. First, it must avoid table rescans, deriving $\hat{H}$ only from H_0 and recent feedback. Second, it must be cheap enough to run near query-planning time. Third, it must degrade gracefully when feedback is sparse or uninformative, preserving the stale prior rather than applying large unsupported corrections.

2.3. Scope

OASIS targets single-column histogram staleness. It does not repair multi-column correlation errors, join-key dependence, physical design problems, or cost-model calibration. These terms remain in the optimizer’s error budget even if the single-column histogram is accurate. The role of OASIS is to improve the marginal statistics that many downstream estimators and optimizers already consume.

3. OASIS Design

3.1. System Overview

OASIS sits between the statistics store and the optimizer. When the optimizer requests a histogram, OASIS reconstructs a canonical quantile representation, retrieves a recent observation window for the column, predicts

corrected quantile boundaries (Stage 1), applies a feedback-consistency projection (Stage 2), and returns an engine-native statistics object to the CBO. Query execution remains unchanged except for lightweight per-conjunct feedback collection.

3.2. Feature Representation

The correction model consumes a fixed tensor containing four blocks. The prior block stores the normalized quantile boundaries of H_0 . The meta block stores the null fraction, observation count ratio, and bucket count. The observation block stores up to K recent predicate observations; each observation contains a predicate-type encoding and numeric features for predicate bounds, estimated selectivity, actual selectivity, and recency. The mask block marks valid observation slots. In our implementation $B = 10$ and $K = 16$, producing a compact input representation.

3.3. Correction Model

OASIS uses a pooled MLP with a prior encoder, an observation encoder, an attention-like pooling layer over the observation window, and a residual output head that predicts corrections to the internal quantile boundaries. Residual prediction encourages conservative behavior when the feedback window is uninformative. The predicted boundaries are clamped to the normalized domain and projected into monotone order to ensure a valid CDF.

The model is trained offline on simulation-derived compound drift. Each training sample contains a stale prior, a causal feedback window, and post-drift quantile targets. Once trained, the same checkpoint is reused across columns in the evaluation; no per-table or per-distribution retuning is used.

3.4. Feedback Collection and Integration

OASIS requires two integration points. A query-completion listener records predicate-level feedback from executor row-count instrumentation. A statistics-assembly hook intercepts the histogram before it reaches the optimizer and substitutes the corrected version. Because the correction is inserted at the statistics layer, the optimizer’s search procedure and cost model remain unchanged.

3.5. Statistics-Format Conversion

Many DBMSs do not expose a standalone editable equi-depth histogram. PostgreSQL, for example, couples histogram boundaries with MCV lists and other per-column summaries. We instantiate the interface for PostgreSQL-style statistics and design it to isolate engine-specific adapters. The adapter has three phases: reconstruct a canonical distribution view from the native object, apply correction on the canonical view, and decompose the corrected view back into native statistics fields. This paper validates the PostgreSQL-style MCV+histogram instantiation; other engines require their own adapter logic.

3.6. Stage 2: Feedback-Consistency Projection

The learned stage (Stage 1) predicts a global marginal correction from learned drift patterns; we call this stage-one-only output OASIS-noProj. For planner use, however, the corrected CDF must also be reliable near the predicates that have just been observed. This local requirement is especially important before multi-column composition, where a small one-dimensional CDF mismatch can become a larger joint-selectivity error. The second stage therefore applies a lightweight projection after the Stage-1 prediction. The projection uses the OASIS-noProj histogram as the starting point and adjusts it to satisfy the current feedback constraints under an ISOMER/IPF-style consistency principle. This step remains a single-column calibration layer; it does not estimate multi-column dependence.

Hard projection.. The default Stage-2 operator builds the interval partition induced by the active feedback predicates, initializes cell masses from the OASIS-noProj histogram, and performs cyclic I-projections so that every active predicate’s interval mass matches its observed selectivity exactly. When sequential drift makes the active set inconsistent, the oldest constraints are dropped first, an implicit stale-feedback filter. We denote the resulting two-stage output OASIS; it is the safe form for downstream composition and the DBMS planner.

Consistency-gated soft projection.. Exact matching is not always optimal. On *future* single-column predicates, matching a handful of recent feedback intervals exactly can overfit them; a softer calibration that regularizes toward the learned marginal generalizes better. We therefore add a soft Stage-2 operator that minimizes $\text{KL}(p \parallel p_{\text{noProj}}) + \lambda \sum_j w_j (A_j p - y_j)^2$ over the same

partition, where A_j is a feedback interval mask and y_j its observed selectivity. The crucial component is the weight w_j : rather than discarding old feedback by age, we discard it by *inconsistency with the current data state*. We fit a reference distribution to the most recent observations, score each constraint by its residual against that reference, and set $w_j = \exp(-(\text{conflict}_j/\tau)^2)$, so feedback that the recent observations contradict is suppressed while old-but-still-consistent feedback is retained. This keeps the single-column generalization benefit of soft calibration without softly fitting stale, contradicted constraints.

A negative result: banded projection.. A natural alternative is to relax the equality constraints into confidence bands $y_j \pm \delta_j$ and I-project the learned prior onto that polytope, so that $\delta_j = 0$ recovers hard projection. This does not help: widening the band monotonically worsens single-column accuracy (Section 5.4). The soft operator’s advantage comes from a better-conditioned cell-level optimization—it attains *lower* feedback residual than hard projection on single-column data—not from relaxed matching, so relaxation is the wrong axis.

The deployed Stage 2: a calibrated router.. No single operator dominates: hard projection is best on the join kernel and the planner, soft projection is best on future single-column and optimizer-facing predicates. We resolve this with a residual-gated *calibrated router* that selects, per case, the candidate with the lowest feedback residual on the current observation window—a deployment-visible signal with no fresh labels—among the stale prior, ISOMER, OASIS-noProj, hard projection, and consistency-gated soft projection. Because the gate is a residual minimizer over a pool that contains hard projection, the router can only improve on the residual signal, and it reduces to hard projection/ISOMER on workloads (joins, the PostgreSQL planner) where soft projection has higher residual. The output of this router is the deployed OASIS; the simpler Hybrid over stale, ISOMER, OASIS-noProj, and OASIS remains available where the soft candidate is unnecessary.

4. Experimental Methodology

The evaluation asks five questions. First, does the statistics-format conversion preserve probability mass and selectivity estimates? Second, how much does OASIS improve single-column histogram quality under drift? Third, when do corrected marginals compose safely with downstream multi-column

estimators, and when is feedback consistency required? Fourth, do corrected statistics affect a real PostgreSQL optimizer pipeline in the intended direction, without measuring runtime? Fifth, what safety checks are needed when feedback is sparse, noisy, or trace-like rather than drawn from the training generator?

Unless otherwise stated, OASIS is trained once on Gaussian compound drift with $q \in \{1, 3, 5, 10, 15, 20\}$ and evaluated on held-out drift intensities $q \in \{1, 3, 5, 10, 15, 20, 25, 30\}$. All feedback-driven methods consume the same observation budget $K = 16$ and emit corrected quantiles on the same $B = 10$ grid. We compare against the stale prior, STHoles [1], ISOMER [2], QuickSel-H [3], and simple feedback heuristics. Metrics include Q-error, selectivity MAE, quantile MAE, feedback residual, PostgreSQL estimated-row Q-error, and plan-shape match to the fresh-statistics planner. For deployment-sensitivity diagnostics that require many perturbed runs, we also use a transparent optimizer-decision proxy: future predicates are generated from held-out distributions, each statistics state selects scan and join choices under a fixed cost proxy, and the selected choice is scored by its true proxy cost. This proxy is used only to analyze feedback-budget and feedback-noise behavior; it is not a runtime benchmark.

5. Single-Column Statistics Correction

5.1. Format-Conversion Validation

The PostgreSQL-style MCV+histogram conversion preserves total probability mass exactly in the tested settings. Round-trip residual-quantile MAE is at machine precision, with mean 2.4×10^{-17} and maximum 9.2×10^{-17} . Selectivity estimates remain consistent before and after conversion, with global MAE 1.1×10^{-4} and worst-case error below 0.39%. With PostgreSQL-like defaults, the full round trip completes in 0.066–0.073 ms. This validates the PostgreSQL-style conversion as a low-overhead bridge between native statistics and the canonical OASIS view.

5.2. Q-Error Across Drift Intensity

Figure 1 reports the main single-column result. Because direct single-column evaluation has no downstream composition to violate, this section isolates the learned repair stage (OASIS-noProj); the full two-stage OASIS is evaluated where the projection matters, in Sections 6.3–6.6, and compared on single-column accuracy in Table 4. At mild drift ($q = 1$), ISOMER

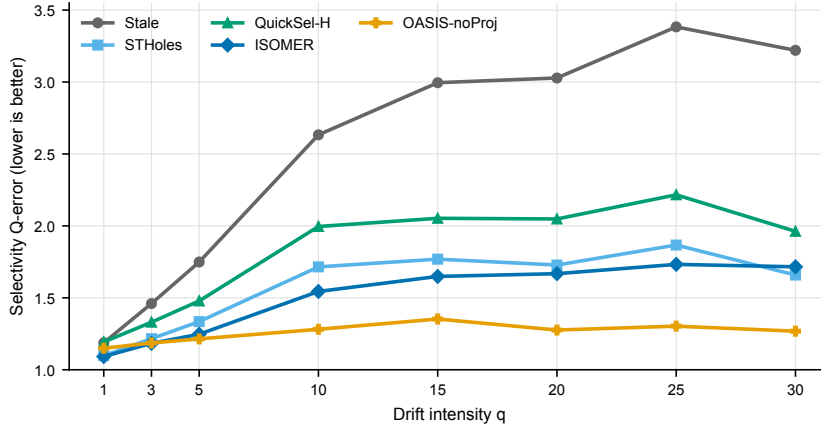


Figure 1: Single-column selectivity Q-error across drift intensity. All methods consume the same feedback budget. This figure isolates the learned repair stage (OASIS-noProj), which remains close to the stale prior at mild drift and becomes strongest as drift grows; the full two-stage OASIS is compared in Table 4.

is strongest and OASIS-noProj remains close to the stale prior, consistent with the graceful-degradation objective. From $q \geq 5$, OASIS-noProj is the best method under the shared feedback budget. At $q = 10, 20$, and 30 , it reduces Q-error over the stale prior by 51.3%, 57.9%, and 60.6%, respectively, outperforming ISOMER in this moderate-to-large drift regime. ISOMER is constrained by the observed feedback window rather than a learned global drift prior. The advantage persists on unseen drift intensities $q = 25$ and $q = 30$.

Table 1 compares OASIS-noProj with simple feedback heuristics. Fixed linear interpolation and exponential feedback averaging are substantially weaker than learned correction; FeedAvg worsens estimation in all tested settings. This shows that the gains are not simply due to using feedback, but to learning how the prior and observations should interact.

5.3. Structural Metrics and Initial-Distribution Generalization

Table 2 shows that OASIS-noProj improves structural histogram quality, not only Q-error. At $q = 10$, quantile MAE drops from 0.208 for the stale prior to 0.055 for OASIS-noProj, and selectivity MAE drops from 0.190 to 0.049. OASIS-noProj remains the best method on both metrics for every $q \geq 5$.

Table 1: Simple heuristic baselines vs. OASIS-noProj (the learned stage-1 repair) at selected drift intensities. Results are averaged over three independent training seeds (42, 123, 456) with a different evaluation protocol than the main single-column comparison; absolute Q-Error values are therefore not directly comparable across the two protocols. Both heuristics consume the same observation window $K=16$. FeedAvg worsens estimation at all drift levels.

q	Q-Error ($\downarrow$ better)			
	Stale	LinInterp	FeedAvg	OASIS-noProj
5	1.791	1.628	1.835	1.238
10	2.471	2.118	2.565	1.255
20	3.370	2.749	3.553	1.296
30	3.500	2.823	3.690	1.345

Table 2: Quantile MAE and Selectivity MAE across drift intensities from the clean synthetic rerun. Bold denotes best method per metric.

q	Quantile MAE ($\downarrow$)					Selectivity MAE ($\downarrow$)				
	Prior	STHoles	QSel-H	ISOMER	OASIS-noProj	Prior	STHoles	QSel-H	ISOMER	OASIS-noProj
1	0.043	0.027	0.036	0.023	0.041	0.038	0.024	0.031	0.020	0.035
3	0.106	0.064	0.071	0.047	0.048	0.095	0.057	0.064	0.040	0.042
5	0.145	0.085	0.094	0.057	0.054	0.131	0.077	0.085	0.051	0.048
10	0.208	0.115	0.126	0.080	0.055	0.190	0.105	0.114	0.072	0.049
15	0.230	0.119	0.128	0.094	0.062	0.203	0.104	0.113	0.083	0.054
20	0.237	0.115	0.127	0.102	0.052	0.214	0.104	0.115	0.091	0.046
25	0.248	0.116	0.131	0.095	0.053	0.223	0.104	0.118	0.084	0.046
30	0.231	0.103	0.111	0.103	0.049	0.204	0.091	0.098	0.090	0.042

Table 3 evaluates transfer across initial distributions at $q = 10$. The learned stage (OASIS-noProj), trained on Gaussian-mixture initial states, remains best across uniform, skewed, bimodal, triangular, exponential, and Gaussian-mixture starts, reducing Q-error by 44%–61% over the stale prior.

The single-column results so far isolate Stage 1. Table 4 adds the deployed two-stage OASIS. Applying the Stage-2 projection costs only a small amount of single-column accuracy relative to OASIS-noProj (aggregate Q-error 1.307 versus 1.254), because direct single-column evaluation has no downstream estimator to violate. At mild drift $q = 1$, ISOMER remains lower (1.092 versus 1.111), but from $q \geq 3$ the full OASIS stays below ISOMER. Thus the deployed form preserves a learned single-column advantage in the moderate-to-large drift regime; this small projection cost is the price

Table 3: Q-Error across different initial distributions at $q=10$. A model trained only on Gaussian-mixture initial states remains best on all tested distributions.

Initial Distribution	Stale	STHoles	QuickSel-H	ISOMER	OASIS-noProj
Gaussian Mixture	2.497	1.453	1.845	1.493	1.262
Uniform	2.409	1.503	1.977	1.534	1.284
Skewed (Power-law)	3.517	1.679	2.202	1.609	1.379
Bimodal	2.488	1.554	2.069	1.462	1.276
Triangular	2.225	1.420	1.833	1.505	1.238
Exponential	3.097	1.592	2.141	1.593	1.292

Table 4: Single-column selectivity Q-error ($\downarrow$) of the full two-stage OASIS versus the OASIS-noProj ablation, on in-distribution compound drift. At mild drift $q = 1$, ISOMER is lowest; from $q \geq 3$, the full OASIS stays below ISOMER, so the deployed (projected) form preserves a learned single-column advantage in the moderate-to-large drift regime. The projection costs only a small amount relative to the stage-1 ablation, the price paid for the downstream composition and join safety shown in Tables 8, 9 and 10.

q	Stale	ISOMER	OASIS-noProj	OASIS	OASIS red. %
1	1.188	1.092	1.150	1.111	+6%
3	1.460	1.184	1.186	1.178	+19%
5	1.749	1.247	1.215	1.209	+31%
10	2.633	1.544	1.281	1.393	+47%
15	2.995	1.649	1.353	1.409	+53%
20	3.028	1.668	1.276	1.364	+55%
25	3.383	1.733	1.303	1.412	+58%
30	3.220	1.716	1.268	1.379	+57%

paid for the downstream composition and join safety established in the following sections.

5.4. Stage-2 Calibration Variants and the Calibrated Router

The previous subsection used hard projection. We now study the Stage-2 calibration directly. This experiment reuses the cached drift data and the same OASIS checkpoint, and evaluates each Stage-2 variant on held-out future predicates per case (geometric-mean selectivity Q-error), together with the feedback residual on the observation window, quantile MAE against fresh statistics, and an optimizer-proxy join regret. It is a controlled comparison of calibration operators, not a new training protocol.

Table 5 shows the regime structure. Full-window soft projection has the lowest single-column Q-error (1.376 versus hard 1.442) and the lowest feed-

Table 5: Stage-2 calibration variants on held-out future predicates (cached drift, fixed OASIS checkpoint). Lower is better. Full-window soft has the best raw single-column metrics but is unsafe under sequential drift (Section 5.5); the calibrated router is the best *deployable* variant, recovering most of the single-column gain over hard projection while remaining safe to route.

Stage-2 variant	Sel. QErr	Feedback resid.	Quantile MAE	Join regret
Hard projection (OASIS-Proj)	1.442	0.0440	0.0519	1.0344
Full-window soft	1.376	0.0348	0.0483	1.0284
Recent-window soft ($k=8$)	1.408	0.0404	0.0471	1.0322
Conflict-aware soft	1.401	0.0392	0.0473	1.0313
Hybrid (no soft)	1.429	0.0395	0.0513	1.0326
Calibrated router	1.395	0.0358	0.0486	1.0302

back residual, confirming that softening helps single-column generalization rather than hurting it. But full-window soft is unsafe under sequential drift: it softly fits stale, contradicted feedback, which (in separate runs) inflates out-of-distribution batch-load Q-error to 1.482 versus hard 1.272. A fixed recent window ($k=8$) removes that failure but discards old-but-still consistent feedback, giving back accuracy (1.408). The consistency-gated soft variant keeps old feedback that the recent observations do not contradict and reaches 1.401 at a lower feedback residual than the fixed window. The calibrated router, selecting per case by feedback residual over {stale, ISOMER, OASIS-noProj, hard, conflict-aware soft}, is best overall at 1.395 with the lowest feedback residual (0.0358), lowest join regret (1.0302), and lowest new-risk loss (1.37% versus hard 1.81%); it routes to soft on 44% of cases and to hard/ISOMER/OASIS-noProj otherwise. As a negative control, a banded projection that relaxes the equality constraints into confidence bands $y_j \pm \kappa \sqrt{y_j(1 - y_j)}$ monotonically worsens single-column Q-error as κ grows (1.437, 1.450, 1.470, 1.509 for $\kappa \in \{0.04, 0.10, 0.20, 0.35\}$), so the gain comes from per-case routing, not from relaxed matching.

The same ordering holds on the optimizer-decision proxy of Section 4: selectivity Q-error / join regret are 1.446/1.0348 for hard projection, 1.404/1.0312 for conflict-aware soft, 1.430/1.0326 for the stale/ISOMER/noProj/hard Hybrid, and 1.396/1.0297 for the calibrated router—the best non-fresh result. Crucially, the router does not trade away safety: where soft projection has higher feedback residual it is never selected. On the FactorJoin kernel the

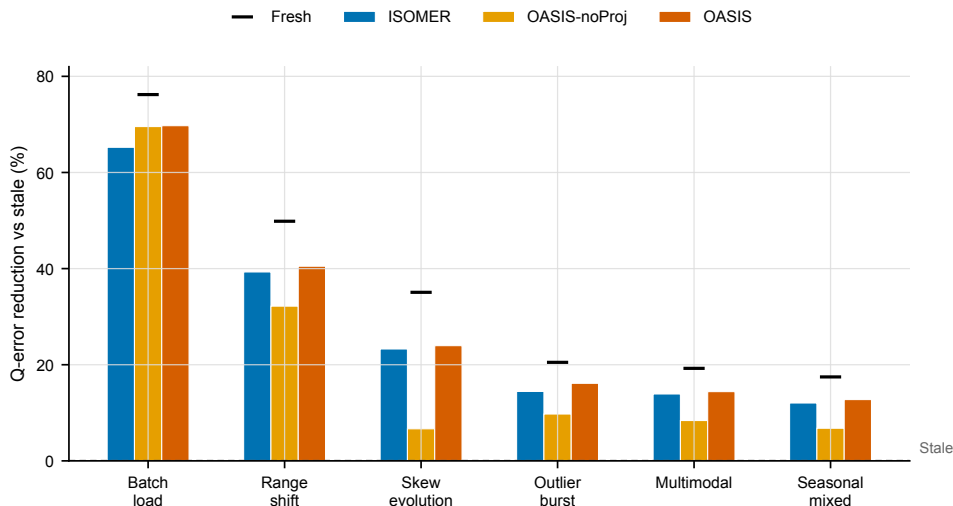


Figure 2: Out-of-distribution drift realism suite. OASIS is trained only on compound drift and evaluated unchanged on deployment-inspired drift families. Bars report the percentage reduction in geometric-mean selectivity Q-error relative to stale statistics for the same drift family; higher is better. The dashed zero line is stale, and black ticks show the fresh post-drift reference.

residual-gated router stays at hard level (join Q-error 1.0174 versus hard 1.024) while soft projection alone is worse (1.0367); and on the PostgreSQL plan-shape batch (Section 6.6) the router is identical to the Hybrid. The calibrated router is therefore $\geq$ hard projection on single-column and optimizer-facing accuracy and equal to the deployed Hybrid on join and planner safety.

5.5. Out-of-Distribution Drift Realism

The main OASIS checkpoint is trained on compound drift. To test whether the learned correction is useful beyond that generator, we evaluate it unchanged on six deployment-inspired drift families: batch loads, range shifts, skew evolution, outlier bursts, multimodal drift, and seasonal/mixed drift. Each case starts from a stale histogram, interleaves the new drift family with 16 feedback observations, and evaluates future selectivity against the final post-drift distribution. No OOD retraining is used.

Figure 2 shows that the correction path remains useful under these unseen drift mechanisms. The strongest OOD stress case is batch loading, where stale Q-error is 4.203; OASIS-noProj reduces it to 1.280, outperforming ISOMER’s 1.461, and the full OASIS further improves to 1.272. On range shift,

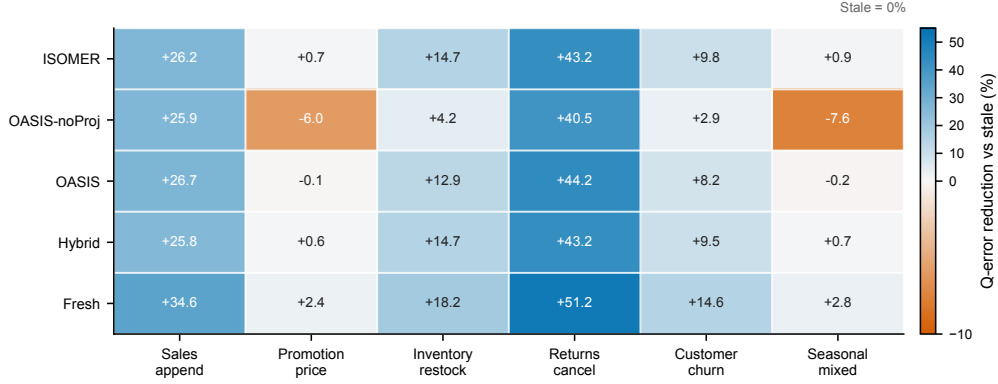


Figure 3: Benchmark-inspired DML trace sanity check. Each trace emits explicit insert/update/delete events anchored to an analytical benchmark maintenance pattern. Cells report the percentage reduction in geometric-mean selectivity Q-error relative to stale statistics for the same trace; higher is better, while negative values indicate regressions below stale. Stale is 0% by definition, and the Fresh row shows the post-drift reference.

skew evolution, outlier, multimodal, and seasonal drift, OASIS is the best non-fresh method, with Q-error between 1.057 and 1.187. This experiment is not a substitute for production trace validation, but it shows that the gains are not confined to the compound-drift generator used for training.

5.6. Benchmark-Inspired DML Trace Sanity Check

The OOD suite changes the drift family, but it still uses synthetic drift operators. We therefore add a second realism check with explicit DML event streams anchored to analytical benchmark table/column patterns: append-heavy sales dates, promotion price revisions, inventory restocking, returns and cancellations, customer segment churn, and seasonal mixed maintenance. Each trace emits insert, update, and delete counts before a feedback observation is collected. These traces are not production logs or DBMS telemetry; they are a transparent sanity check that the correction layer remains useful when drift is produced by event sequences rather than by the compound training generator.

Figure 3 reinforces the same deployment pattern under event-sequence drift. Across the six traces, stale statistics have aggregate Q-error 1.295; the full OASIS reduces this to 1.076 and residual-gated Hybrid to 1.072. OASIS is the best non-fresh method on sales append and returns/cancellation traces, where the stale histogram is visibly shifted. On the milder price, inventory,

Table 6: Counterfactual selectivity evaluation on compound drift test data. “Sampled-Refresh” is a simulated column-level resampling proxy. Recovery measures the fraction of fresh/oracle Q-error improvement recovered by the learned stage (OASIS-noProj). No query runtime is measured.

q	#Pred	Stale QE	Sampled-Refresh	OASIS-noProj QE	Recovery
5	30	1.714	1.492	1.263	69%
10	30	2.720	2.121	1.232	77%
15	30	2.798	2.183	1.284	74%
20	30	2.779	2.201	1.389	64%
Mean	120	2.503	1.999	1.292	71%

customer, and seasonal traces, ISOMER is best or tied because enforcing the current feedback window is sufficient. OASIS-noProj can regress on these mild update-heavy traces, which supports the two-stage design: the learned stage is useful, but the statistics object passed to a planner should be the projected (full) OASIS or residual-gated Hybrid.

6. Optimizer-Facing Evaluation

6.1. Counterfactual Selectivity Evaluation

We first isolate the causal effect of the statistics state on selectivity estimation. The same held-out drift samples are evaluated under stale, OASIS-noProj-corrected, and simulated sampled-refresh statistics, while the underlying post-drift distribution and predicates are held fixed. This is not a runtime benchmark; it measures selectivity Q-error under controlled statistics states.

Table 6 shows that OASIS-noProj recovers 64%–77% of the fresh/oracle improvement and substantially outperforms the sampled-refresh proxy. The result should be interpreted as per-predicate selectivity evidence, not as evidence of wall-clock query acceleration.

6.2. Does the Learned Stage Help After Projection?

A reasonable concern about the two-stage design is whether the learned stage contributes anything once the projection is applied: in several aggregate downstream tables the full OASIS and ISOMER are close. We isolate the question with a projection-initialization ablation. We hold the projection operator and the feedback constraints fixed and vary only the marginal that

Table 7: Projection-initialization ablation on held-out future predicates (in-distribution compound drift; geometric-mean selectivity Q-error). The projection operator and feedback constraints are identical across columns; only the marginal that initializes the projection changes. **Proj(stale)** initializes from the stale histogram and is exactly ISOMER; **Proj(OASIS)** initializes from the learned stage and is the full two-stage OASIS; **Proj(fresh)** initializes from the post-drift marginal as a reference upper bound. Win frac is the fraction of held-out predicates on which the full OASIS beats ISOMER.

q	Stale	Proj(stale)	OASIS-noProj	Proj(OASIS)	Proj(fresh)	Fresh	Proj(OASIS) vs ISOMER	Win frac
1	1.224	1.137	1.271	1.181	1.104	1.000	-3.9%	43.3%
3	1.679	1.298	1.370	1.283	1.146	1.000	+1.2%	53.5%
5	2.004	1.371	1.511	1.348	1.160	1.000	+1.7%	55.0%
10	2.774	1.568	1.647	1.461	1.169	1.000	+6.8%	59.1%
15	3.103	1.735	1.894	1.616	1.167	1.000	+6.8%	56.6%
20	3.124	1.784	1.827	1.590	1.165	1.000	+10.9%	60.9%
25	3.380	1.763	1.911	1.601	1.173	1.000	+9.2%	58.3%
30	3.091	1.790	1.869	1.556	1.174	1.000	+13.0%	60.3%
All	2.415	1.536	1.645	1.446	1.157	1.000	+5.9%	55.9%

initializes the projection: initializing from the stale histogram reproduces ISOMER, initializing from the learned stage gives the full OASIS, and initializing from the post-drift marginal is a reference upper bound. All variants are scored on held-out future predicates, not on the feedback window they were fit to.

Table 7 shows that the learned stage is not decorative. Holding the projection fixed, initializing from the learned marginal lowers future-predicate Q-error from 1.536 (ISOMER) to 1.446 (full OASIS), a 5.9% reduction, and the advantage grows monotonically with drift: +6.8%, +10.9%, and +13.0% at $q = 10, 20$, and 30 , while the two methods are within noise at $q = 1$. Per-predicate, the full OASIS beats ISOMER on 55.9% of held-out predicates, up to 60.9% at $q = 20$. A threshold-only predicate population gives the same pattern more mildly (+4.3% overall, +8.8% at $q = 30$).

Two further readings support the claim that both stages are necessary. First, the columns form a clean initialization-quality ladder—stale 1.536, learned 1.446, fresh 1.157—so a better starting marginal yields a better projected result, and the learned initialization closes about a quarter of the ISOMER-to-fresh headroom. Second, neither stage alone reaches the two-stage result: the raw learned marginal (OASIS-noProj, 1.645) is worse than ISOMER on this metric, and ISOMER (projection from a stale start) is worse than the full OASIS. This also explains why aggregate downstream tables show ISOMER and the full OASIS closer: the learned stage’s mea-

Table 8: Marginal-to-joint composition: average Q-error ($\downarrow$) for two-column range predicates. Values average configuration-level geometric means over 20 correlation/drift settings. OASIS is the full two-stage system; OASIS-noProj is the learned stage without the feedback-consistency projection; Hybrid uses only feedback-window residuals for selection.

Estimator	Stale	OASIS-noProj	ISOMER	OASIS	Hybrid	Fresh	OASIS-noProj +%	OASIS +%	Hybrid +%
Independence / max-entropy	1.820	1.679	1.220	1.222	1.214	1.185	+7.9%	+31.8%	+32.2%
Gaussian copula	1.558	1.513	1.185	1.176	1.166	1.139	+2.4%	+23.1%	+23.8%

surable advantage is on the marginal’s future-predicate accuracy, and how much of it survives depends on how strongly a given downstream estimator’s error is dominated by the marginal. A finer locality decomposition is given in Appendix Appendix A.

6.3. Marginal-to-Joint Composition

This experiment studies composition rather than replacing multi-column cardinality-estimation methods. OASIS repairs one-dimensional marginals, so the question is how those repaired marginals behave when a downstream estimator combines them under a fixed dependence assumption. We generate correlated two-column tables with drift, correct each single-column marginal independently, and feed the marginals into two joint estimators: independence, the maximum-entropy estimate under only one-dimensional marginal constraints, and a Gaussian copula with fixed correlation structure. Fresh uses post-drift marginal inputs under the same downstream estimator; it is not an oracle joint distribution.

Table 8 gives the main diagnostic result. Despite the strong single-column performance of the learned stage, OASIS-noProj provides only modest and unstable gains after two-column composition: it improves average joint Q-error by 7.9% under independence and by 2.4% under the Gaussian copula. ISOMER is substantially stronger in this setting because it enforces current feedback constraints. The attribution explains the gap: OASIS-noProj improves marginal Q-error over stale in only 43.6% of composition rows and beats ISOMER on marginal Q-error in only 3.9%. Thus, the learned stage should not be interpreted as a stable multi-column correction method on its own.

The two-stage design resolves this directly: the projection stage calibrates the learned marginal before composition, without changing the dependence model. By projecting the OASIS-noProj marginal onto the feedback constraints, the full OASIS reaches Q-error 1.222 under independence and 1.176

Table 9: OASIS as a drop-in marginal upgrade across a family of composition estimators. Average joint Q-error ($\downarrow$) for two-column range predicates; dependence structure is held fixed per estimator and only the marginal input varies. The full two-stage OASIS and Hybrid improve every estimator, whereas OASIS-noProj (the learned stage without the feedback-consistency projection) is weak and unstable, confirming the projection is required across composition methods.

Estimator	Stale	OASIS-noProj	ISOMER	OASIS	Hybrid	Fresh	OASIS-noProj +%	OASIS +%	Hybrid +%
Independence	1.924	1.798	1.191	1.191	1.185	1.138	+6.6%	+38.1%	+38.4%
IPF/Sinkhorn-2D	1.668	1.564	1.302	1.280	1.274	1.248	+6.3%	+23.3%	+23.6%
Gaussian copula	1.645	1.558	1.251	1.224	1.216	1.182	+5.3%	+25.6%	+26.1%
Clayton copula	1.836	1.571	1.310	1.256	1.251	1.222	+14.4%	+31.6%	+31.9%
Gumbel copula	1.639	1.535	1.254	1.234	1.226	1.192	+6.4%	+24.7%	+25.2%
Frank copula	1.621	1.552	1.291	1.260	1.253	1.221	+4.2%	+22.2%	+22.7%

under the Gaussian copula, nearly matching ISOMER and approaching fresh marginal inputs. Hybrid provides a conservative deployment policy by choosing among stale, ISOMER, OASIS-noProj, and OASIS using only feedback-window residuals.

6.4. Composition-Family Generalization

The two-column study above uses two downstream estimators. A natural concern is whether the deployment pattern—OASIS-noProj is an unstable composition input, and the feedback-consistency projection is required—is specific to those two estimators or general across composition methods that consume single-column marginals. We therefore embed OASIS as a drop-in marginal source in a family of six independently designed composition estimators and vary only the marginal input. The family covers the independence/maximum-entropy estimator, iterative proportional fitting (IPF/Sinkhorn) on a two-dimensional grid, and four copulas with distinct dependence structure: Gaussian, Clayton, Gumbel, and Frank. To keep the comparison a clean marginal ablation and to respect the scope boundary that OASIS does not infer dependence, the dependence structure is held fixed and identical for every marginal input: each copula uses one association parameter derived from the data-generating correlation, and IPF seeds its cell-level association from a stale two-dimensional sketch and only reconciles it with each method’s corrected one-dimensional marginals. The Archimedean copulas are intentionally mis-specified models for the Gaussian-correlated data, which tests whether the marginal-repair benefit survives an imperfect dependence model.

Table 9 confirms both halves of the deployment story across the whole family. OASIS-noProj again gives only weak and inconsistent composition

Table 10: OASIS embedded in the FactorJoin bin-based join kernel. Join-cardinality Q-error ($\downarrow$) for a two-table equi-join whose join-key distributions drift; only the single-column key histogram consumed by FactorJoin varies across methods. The full two-stage OASIS and Hybrid recover most of the stale-to-fresh join error, whereas OASIS-noProj (the learned stage without projection) is actively harmful—it exceeds the stale baseline in this bilinear kernel.

q	Stale	OASIS-noProj	ISOMER	OASIS	Hybrid	Fresh	OASIS +%	Hybrid +%
5	1.323	1.248	1.009	1.010	1.008	1.005	+23.6%	+23.8%
10	1.179	1.435	1.013	1.031	1.021	1.005	+12.6%	+13.4%
20	1.179	1.447	1.017	1.021	1.021	1.005	+13.4%	+13.4%
30	1.178	1.389	1.026	1.034	1.020	1.005	+12.3%	+13.4%
All	1.213	1.377	1.016	1.024	1.017	1.005	+15.6%	+16.1%

gains (between +4.2% and +14.4% over stale), whereas the full OASIS improves every estimator by 22.2%–38.1% and Hybrid by 22.7%–38.4%, in each case approaching the fresh-marginal input. The gain therefore does not depend on any single dependence model, and it persists even under the misspecified Archimedean copulas. This generalizes the calibration finding from two estimators to a method family: the learned stage is a useful input to existing composition estimators, but the feedback-consistency projection is what makes it a stable one.

6.5. FactorJoin Integration

The previous composition studies remain single-table. To test whether corrected marginals propagate into a learned *join* cardinality estimator, we embed OASIS in the bin-based join kernel of FactorJoin [4]. For an equi-join $A.k = B.k$, FactorJoin partitions the join-key domain into bins and estimates the join size as $\sum_{\text{bin}} \text{cnt}_A[\text{bin}] \text{cnt}_B[\text{bin}] / \text{dv}[\text{bin}]$, where each table’s per-bin key frequency $\text{cnt}_T[\text{bin}]$ is read from a single-column histogram of its join-key distribution. We reimplement this kernel and make the per-table key histogram the OASIS-correctable object: when the key distribution drifts, a stale histogram yields wrong per-bin frequencies and a wrong join estimate. The two tables’ keys are generated independently, so the only quantity that varies across methods is the quality of each single-column key histogram; the binning, the distinct-values-per-bin assumption, and the row counts are identical for all methods. The metric is join-cardinality Q-error against the exact join size on the drifted data; no query runtime is measured.

Table 10 shows the strongest form of the calibration finding, and Figure 4 visualizes the deployable calibrated methods on a stale-to-fresh-normalized

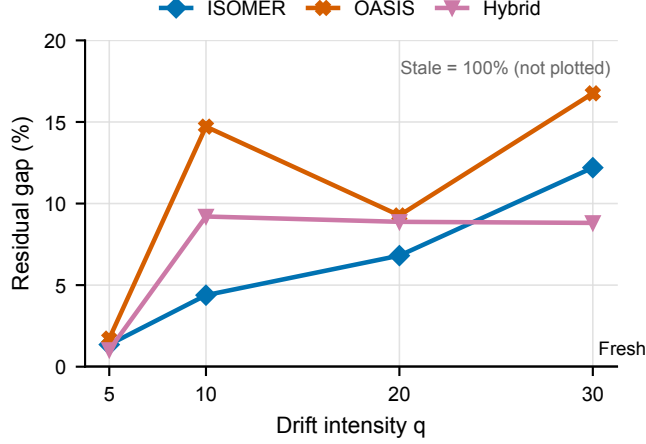


Figure 4: FactorJoin join-cardinality Q-error by drift intensity for the deployed/calibrated marginal sources. Values are the residual gap to the fresh-marginal floor, normalized by the stale-to-fresh gap; 0% is fresh and 100% is stale, so lower is better. The no-projection ablation is reported in Table 10; the full OASIS and Hybrid stay near the fresh-marginal floor.

scale. In Table 10, OASIS-noProj does not merely underperform—it is actively harmful, raising aggregate join Q-error above the stale baseline (-13.5% , and as much as -22.7% at moderate drift). The mechanism is structural: the FactorJoin estimate is a bilinear functional of the two per-bin mass vectors, so it is sensitive to mass concentration rather than to coarse range selectivity. The learned stage, trained to minimize range-selectivity error, can return a directionally correct but over-concentrated histogram—occasionally with collapsed boundaries after monotone clamping—whose fine-grained mass profile inflates $\sum_{\text{bin}} p_A p_B$ and therefore the join estimate. The feedback-consistency projection mitigates this failure: the full OASIS improves over stale at every tested drift level (12.3%–23.6%, with 15.6% aggregate gain), Hybrid improves by 13.4%–23.8%, and both closely track the fresh-marginal input. The effect is stable as the join-bin count varies from 10 to 100. The broader lesson sharpens the deployment boundary: the more non-linear the downstream composition, the more essential feedback-consistency calibration becomes, and a join estimator should consume the full OASIS rather than the raw learned marginal.

Table 11: Multi-configuration PostgreSQL planner-only statistics-injection evidence. Values are means over configurations; parentheses show one standard deviation across configurations. True rows use `COUNT(*)`; estimated rows and plan shapes use `EXPLAIN (FORMAT JSON)`. No query runtime is measured.

Method	Row QE	Fresh Plan	Recovery	New Deviations
Stale	28.436 (6.585)	56.0% (5.5)	0.0% (0.0)	0.0% (0.0)
ISOMER	2.372 (0.283)	96.1% (1.9)	92.2% (3.9)	1.0% (1.0)
OASIS-noProj	3.313 (0.228)	88.6% (2.7)	91.4% (3.2)	13.7% (2.0)
OASIS	2.375 (0.275)	96.1% (1.9)	92.2% (3.9)	1.0% (1.0)
Hybrid	2.374 (0.286)	96.1% (1.9)	92.2% (3.9)	1.0% (1.0)
Fresh	1.415 (0.095)	100.0% (0.0)	100.0% (0.0)	0.0% (0.0)

6.6. PostgreSQL Planner-Only Statistics Injection

The previous experiments evaluate selectivity and downstream estimator behavior. We next validate that corrected statistics can affect a real DBMS optimizer pipeline. The PostgreSQL experiment creates a synthetic table with fresh current data, builds alternative single-column statistics for `fact.x`, and injects them into `pg_statistic` by copying typed statistics rows from analyzed source tables. For each query, true cardinality is obtained with `COUNT(*)`; PostgreSQL estimated rows and plan shape are obtained with `EXPLAIN (FORMAT JSON)`. The experiment does not measure query runtime.

Table 11 provides the strongest DBMS-facing evidence in this paper. The batch covers two drift directions, two table sizes, and three random seeds, yielding 12 PostgreSQL configurations. Across all query instances, stale statistics produce aggregate row Q-error 27.748 and match the fresh-statistics plan shape on only 56.0% of queries. OASIS-noProj reduces row Q-error to 3.306 and recovers 91.7% of stale/fresh plan-shape disagreements, demonstrating that learned marginal correction propagates into PostgreSQL’s native planner. However, the learned stage alone also introduces new plan deviations in 13.8% of cases where stale statistics already matched fresh.

The full OASIS largely removes this safety issue. It reduces aggregate row Q-error to 2.360, matches the fresh-statistics plan shape on 96.1% of queries, recovers 92.6% of stale/fresh plan-shape disagreements, and reduces new plan deviations to 1.1%. Across configurations, mean row Q-error is 2.375 ± 0.275 for OASIS, 2.372 ± 0.283 for ISOMER, and 3.313 ± 0.228 for OASIS-noProj. The planner-only result supports the two-stage design: the learned stage provides the global marginal correction, while the projection

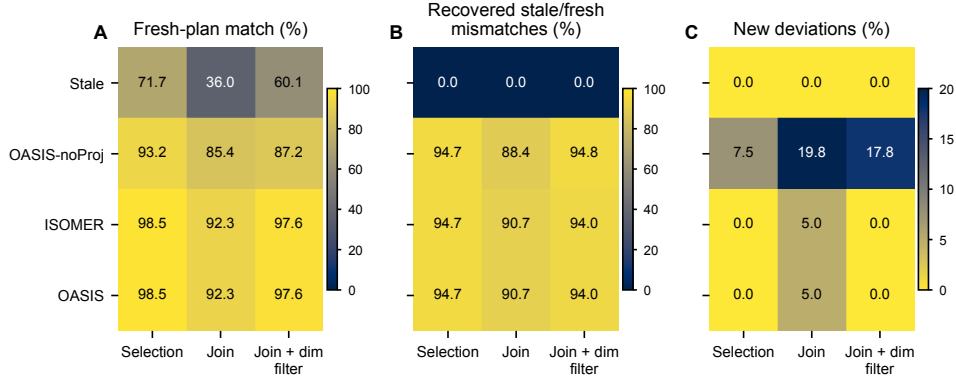


Figure 5: PostgreSQL planner-only plan-shape effects by query family. Cell labels are percentages. Panel A reports fresh-statistics plan-shape match, Panel B reports recovery of stale/fresh mismatches, and Panel C reports new deviations where stale already matched fresh.

stage calibrates it before the DBMS planner consumes it.

Figure 5 breaks the PostgreSQL result down by query family. Stale statistics disagree with the fresh-statistics plan on 95/336 selection queries, 215/336 join queries, and 134/336 join-with-dimension-filter queries. The full OASIS matches the fresh plan on 98.5% of selections, 92.3% of joins, and 97.6% of filtered joins. The safety benefit over the learned stage alone is clearest in new deviations: OASIS-noProj creates new plan-shape deviations in 7.5%, 19.8%, and 17.8% of the three families, whereas the full OASIS reduces these rates to 0.0%, 5.0%, and 0.0%.

Finally, we check that adding the consistency-gated soft projection to the calibrated router introduces no plan-shape regression here. On a six-configuration planner subset (left/right/bimodal shift $\times$ two seeds, 504 queries), the calibrated router is byte-identical to the residual-gated Hybrid—row Q-error 2.234, fresh-plan match 90.1%, new plan deviations 8.17%—because the gate routes to ISOMER on every configuration and never selects soft. For reference, hard projection alone is 2.237/90.9%/6.86% and soft projection alone is 2.277/91.1%/6.54% on the same subset. The residual gate’s preference for ISOMER over hard projection on these planner-facing cases is a property of the existing Hybrid, not of the soft candidate: adding soft to the pool is plan-shape-safe by construction, and the router’s single-column and optimizer-facing gains (Section 5.4) come at no planner cost relative to the deployed Hybrid.

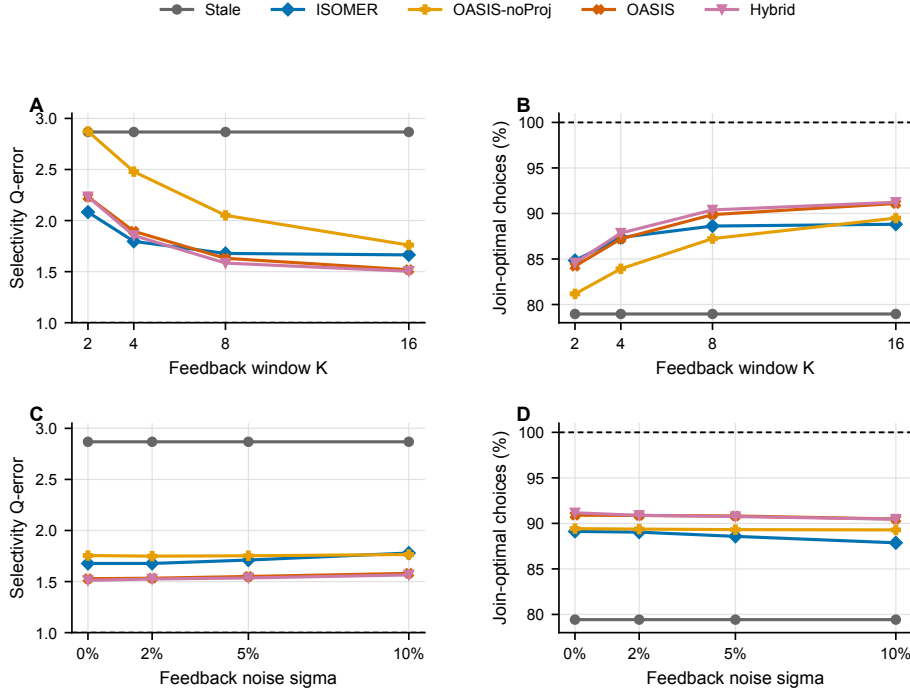


Figure 6: Feedback-budget and feedback-noise sensitivity in the optimizer-decision proxy. Panels A–B vary the visible feedback window K ; Panels C–D perturb feedback selectivities with multiplicative log-normal noise while holding future predicates fixed. Dashed lines denote fresh statistics.

6.7. Feedback Budget and Noise Sensitivity

The deployment form of OASIS depends on two practical conditions: how many recent feedback observations are available and how accurate those feedback observations are. To stress these conditions without running additional DBMS loads, we use the optimizer-decision proxy described in Section 4. All methods see the same feedback window and future predicates; only the feedback budget or feedback values are changed.

Panels A–B of Figure 6 vary the number of visible feedback observations. With only two observations, consistency projection is helpful but limited: the full OASIS reaches selectivity Q-error 2.233 and Hybrid matches 84.6% of optimal join choices, compared with stale Q-error 2.867 and 79.0% join-optimal choices. As the window grows, the learned correction and projection become more complementary. At $K = 16$, the full OASIS reaches Q-error 1.518 and

Table 12: Deployment safety checks. Projection and residual-based Hybrid gating are evaluated without oracle/fresh information. P/I/O/S denote OASIS (full two-stage), ISOMER, OASIS-noProj, and stale choices in the residual gate.

Check	Risk signal	Unguarded state	Calibrated/gated state	Gate signal
PostgreSQL planner	New plan deviations	OASIS-noProj 13.8%	OASIS 1.1%; FreshPlan 96.1%	projection
PostgreSQL joins	Join new deviations	OASIS-noProj 19.8%	OASIS 5.0%; Recovery 90.7%	projection
Sparse feedback ($K = 2$)	Join-optimal choices	Stale 79.0%	Hybrid 84.6%; NewRisk 2.5%	I 27%, P 57%, O 14%, S 1%
Full feedback ($K = 16$)	Join-optimal choices	Stale 79.0%	Hybrid 91.2%; NewRisk 1.4%	I 37%, P 47%, O 14%, S 2%
10% feedback noise	Join-optimal choices	Stale 79.4%	Hybrid 90.5%; NewRisk 1.7%	I 36%, P 42%, O 18%, S 3%
DML trace sanity	Selectivity Q-error	Stale 1.295	Hybrid 1.072; OASIS 1.076	I 77%, P 20%, O 2%, S 1%

Hybrid reaches 1.504, with 91.1%–91.2% join-optimal choices. Hybrid’s selections also show that no single component dominates all cases: at $K = 16$, it selects ISOMER for 37% of cases, the full OASIS for 47%, OASIS-noProj for 14%, and stale for 2%.

Panels C–D perturb feedback selectivities with multiplicative log-normal noise while keeping the future predicates and true selectivities fixed. The diagnostic uses 64 held-out cases per drift intensity and one noise seed. At 10% feedback noise, the full OASIS still improves selectivity Q-error from 2.868 to 1.580 and maintains 90.5% join-optimal choices; Hybrid reaches Q-error 1.565 with the same join-optimal rate. ISOMER is more sensitive to noise in this setting, rising from Q-error 1.677 at clean feedback to 1.779 at 10% noise. The result supports the role of the projection stage as a calibration layer, but it also motivates residual-based gating and abstention when feedback attribution is unreliable.

6.8. Deployment Safety Summary

Table 12 collects the safety evidence that determines how OASIS should be deployed. The learned stage alone (OASIS-noProj) improves row-estimation accuracy, but it can introduce plan deviations when the stale planner already agreed with fresh statistics. The projection stage reduces these new deviations from 13.8% to 1.1% in the PostgreSQL batch and from 19.8% to 5.0% on join queries. In the optimizer-decision proxy, residual-gated Hybrid improves join-optimal choices under sparse feedback, full feedback, and 10% feedback noise while keeping new risky decisions near 1–3%. On the benchmark-inspired DML traces, Hybrid improves aggregate selectivity Q-error from 1.295 to 1.072 and selects ISOMER or the full OASIS in 97% of cases. These results make the projection stage and residual gating part of the method, rather than optional post-processing.

7. Overhead and Deployment Considerations

Per-correction OASIS inference averages 1.06 ms in our implementation, including tensorization and one forward pass. The PostgreSQL-style statistics-format conversion adds 0.066–0.073 ms, for a combined correction cost of approximately 1.13 ms. Feedback collection adds bounded bookkeeping per filter conjunct and emits one observation tuple per conjunct at query completion. These measurements support the intended use case: maintaining optimizer-facing statistics between scheduled statistics refreshes, not replacing full refreshes after major schema changes or bulk data shifts.

8. Related Work

Classical histogram construction builds statistics from data scans [5, 6], while streaming summaries such as KLL sketches provide compact approximate quantiles [7]. Adaptive selectivity estimation has long used query feedback to refine estimation functions [8]. Feedback-driven histograms, including self-tuning histograms, STHoles, and ISOMER, refine statistics from observed query results [9, 1, 2]. OASIS differs by learning a reusable drift-correction function that emits DBMS-consumable statistics rather than applying only local per-table refinement rules or returning point selectivity estimates.

Learned cardinality estimators such as NeuroCard, Naru, DeepDB, FACE, and MSCN predict query cardinalities using learned density or representation models [10, 11, 12, 13, 14]. These systems are complementary to OASIS: they often output point estimates for specific query classes, while OASIS outputs corrected marginal statistics that can be consumed by an existing CBO. Copula-based methods and factorized join estimators such as FactorJoin [4] similarly combine one-dimensional marginals with a dependence or join-key model; our composition-family and FactorJoin experiments embed OASIS as the marginal source in exactly these estimators and show that marginal quality and feedback consistency are prerequisites for such downstream use.

Plan-level optimization systems such as LEO, Neo, and Bao adjust plan choices or optimizer behavior based on execution history [15, 16, 17]. OASIS operates below that layer by correcting the statistics object itself. A system could combine both approaches: statistics correction supplies better base-table inputs, while plan-level learning handles residual cost-model or join-order effects.

9. Limitations

The evaluation is limited to tested synthetic drift families, benchmark-inspired DML traces, and controlled PostgreSQL planner-only scenarios. The DML traces make insert/update/delete event streams explicit, but they are not production logs or DBMS telemetry; validating the same correction policy on production statistics histories and workload traces remains future work. The PostgreSQL batch covers two drift directions, two table sizes, and three random seeds, so it validates the statistics-to-planner path more robustly than a single case, but it does not establish wall-clock runtime improvement. Runtime depends on executor behavior, caching, physical design, system load, and cost-model calibration. OASIS also does not infer multi-column dependence from single-column feedback. When downstream estimates depend on correlations, an external dependence model or multivariate statistics source is still required. Finally, feedback itself can be noisy because of predicate attribution, sampling, or concurrent updates. Our noise diagnostic shows graceful degradation up to 10% multiplicative feedback noise, but deployments should pair correction with residual-based gating, abstention, and rollback policies. The calibrated router also gates on feedback residual, which is a proxy for estimate quality rather than for plan-shape risk: on the PostgreSQL subset it prefers ISOMER and so inherits the Hybrid’s slightly higher new-deviation rate (8.17% versus 6.86% for hard projection alone), even though it never routes to the soft candidate there. A plan-shape-aware gating criterion that could route to hard projection over ISOMER on planner-facing cases is left to future work.

10. Conclusion

OASIS repairs stale single-column optimizer statistics from query feedback without rescanning tables or modifying the optimizer. It is a two-stage pipeline: a compact learned model predicts corrected quantile boundaries from a stale prior and recent observations, and a feedback-consistency projection then calibrates the learned marginal before it is exposed downstream; a PostgreSQL-style adapter connects the canonical correction view to DBMS-native statistics.

The central lesson is that both stages are necessary, and they answer complementary failure modes. The learned stage extrapolates drift to future, unseen predicates by using a learned global drift prior, while local-feedback

methods such as ISOMER are constrained by observed feedback. This drives the single-column gains across synthetic drift, OOD families, and distribution transfer. But a globally plausible learned marginal need not be consistent with the predicates just observed, and downstream estimators amplify that inconsistency: the stage-one marginal gives only weak and unstable multi-column gains and is actively harmful in the bilinear FactorJoin join kernel. The projection stage mitigates this downstream failure, at the cost of a small amount of single-column peak accuracy. A further lesson is that this Stage-2 calibration is regime-dependent: hard projection is best for join and planner safety, while a consistency-gated soft projection—which down-weights feedback contradicted by the most recent observations—generalizes better on future single-column and optimizer-facing predicates. Because no single operator dominates, the deployed Stage 2 is a residual-gated calibrated router that selects per case from a deployment-visible signal; it lowers single-column Q-error from 1.442 to 1.395 and optimizer-proxy selectivity Q-error from 1.446 to 1.396 over hard projection, recovering the projection’s single-column cost, while reducing exactly to hard projection or ISOMER on the join kernel and PostgreSQL planner, where it never routes to soft. A banded relaxation of the equality constraints was a negative control that localizes the gain to routing rather than to relaxed matching. Because OASIS acts at the marginal layer, it embeds as a drop-in marginal source for existing composition estimators, improving every member of a six-estimator family by 22%–38%. In FactorJoin, OASIS-noProj is harmful, while the full OASIS recovers stable join-cardinality gains. In the 12-configuration PostgreSQL planner-only batch, OASIS reduces aggregate row-estimation Q-error from 27.748 to 2.360, matches the fresh-statistics plan shape on 96.1% of queries, and keeps new plan deviations near 1%—where the learned stage alone would introduce them—without measuring or claiming query runtime improvement. A residual-gated Hybrid policy, choosing among stale, ISOMER, OASIS-noProj, and OASIS from the current feedback residual, reduces aggregate trace-grounded Q-error from 1.295 to 1.072. This makes OASIS a practical component for maintaining optimizer-facing statistics between scheduled ANALYZE passes.

Appendix A. Feedback-Locality Diagnostic

To see where the learned stage’s advantage comes from, we bin the held-out future predicates of Section 6.2 by their distance to the feedback window.

Table A.13: Feedback-locality diagnostic on the mixed predicate population. Held-out future predicates are binned by the directed Hausdorff distance from their endpoints to the nearest feedback endpoint in fresh-CDF space. Values are geometric-mean selectivity Q-error; ISOMER is projection from a stale start and OASIS is the full two-stage system. OASIS win is the per-predicate fraction on which the full OASIS beats ISOMER.

Locality	Preds	Stale	ISOMER	OASIS-noProj	OASIS	Fresh	OASIS vs ISOMER	OASIS win
near $\leq$ 0.05	33012	2.159	1.441	1.487	1.352	1.000	+6.1%	58.5%
mid $\leq$ 0.15	21712	2.546	1.574	1.777	1.495	1.000	+5.0%	53.6%
far $>$ 0.15	10812	3.054	1.779	1.918	1.660	1.000	+6.7%	52.2%

Each predicate endpoint is mapped to fresh-CDF space and assigned the directed Hausdorff distance to the nearest feedback endpoint, giving near (≤ 0.05), mid (≤ 0.15), and far (> 0.15) bins.

Table A.13 reports the diagnostic on the mixed predicate population. The full OASIS beats ISOMER in every bin, and—restricting to the operative drift regime $q \geq 5$ —by +9.2%, +6.9%, and +8.0% in the near, mid, and far bins. The advantage is therefore not confined to predicates near the feedback window. The raw learned marginal (OASIS-noProj) is, by contrast, worse than ISOMER in every bin and worst in the far bin, confirming that raw learned marginals are not reliably deployable across locality bins; the projection turns the learned initialization into a stable statistics object.

The far bin is predicate-type dependent. For a threshold-only population (pure $</>$ predicates, which probe only a cumulative CDF level), the far-bin advantage shrinks to +0.8% at $q \geq 5$, while near and mid remain strong (+7.7% and +5.4%). Range and point predicates, which probe local density rather than a cumulative level, retain the far-bin advantage. The learned stage’s value in uncovered regions therefore comes from better local density, which helps range and point queries but adds little to purely cumulative threshold queries.

References

- [1] N. Bruno, S. Chaudhuri, L. Gravano, STHoles: A multidimensional workload-aware histogram, in: Proceedings of SIGMOD, 2001, pp. 211–222.
- [2] V. Markl, N. Megiddo, M. Kutsch, T. M. Tran, P. Lang, V. Raman, Consistent selectivity estimation via maximum entropy, The VLDB Journal 16 (2) (2007) 169–188.

- [3] Y. Park, S. Zhong, B. Mozafari, QuickSel: Quick selectivity learning with mixture models, in: Proceedings of SIGMOD, 2020, pp. 1017–1033.
- [4] Z. Wu, P. Negi, M. Alizadeh, T. Kraska, S. Madden, FactorJoin: A new cardinality estimation framework for join queries, Proceedings of the ACM on Management of Data (SIGMOD) 1 (1) (2023) 1–27.
- [5] Y. E. Ioannidis, V. Poosala, Universality of serial histograms, in: Proceedings of the 19th VLDB Conference, 1993, pp. 256–267.
- [6] V. Poosala, Y. E. Ioannidis, Selectivity estimation without the attribute value independence assumption, in: Proceedings of the 23rd VLDB Conference, 1997, pp. 486–495.
- [7] N. Ivkin, J. Nelson, H. Yu, V. Braverman, KLL: Approximately optimal streaming quantile estimation, Proceedings of the ACM on Management of Data 2 (1) (2019) 1–23.
- [8] C. M. Chen, N. Roussopoulos, Adaptive selectivity estimation using query feedback, in: Proceedings of the 24th ACM SIGMOD International Conference on Management of Data, ACM Press, 1994, pp. 161–172. doi:10.1145/191839.191874.
- [9] A. Aboulmaga, S. Chaudhuri, Self-tuning histograms: Building histograms without looking at data, in: ACM SIGMOD Record, Vol. 28, 1999, pp. 181–192.
- [10] Z. Yang, A. Kamsetty, S. Luan, E. Liang, Y. Duan, X. Chen, I. Stoica, NeuroCard: One cardinality estimator for all tables, in: Proceedings of the VLDB Endowment, Vol. 14, 2020, pp. 61–73.
- [11] Z. Yang, E. Liang, A. Kamsetty, C. Wu, Y. Duan, X. Chen, P. Abbeel, J. M. Hellerstein, S. Krishnan, I. Stoica, Deep unsupervised cardinality estimation, Proceedings of the VLDB Endowment 13 (3) (2019) 279–292.
- [12] B. Hilprecht, A. Schmidt, M. Kulesa, A. Molina, K. Kersting, C. Binnig, DeepDB: Learn from data, not from queries!, in: Proceedings of the VLDB Endowment, Vol. 13, 2020, pp. 992–1005.

- [13] Z. Yang, E. Liang, Y. Duan, S. Luan, I. Stoica, J. M. Hellerstein, FACE: A normalizing flow based cardinality estimator, in: SIGMOD Workshop on Approximate Query Processing, 2020.
- [14] A. Kipf, T. Kipf, B. Radke, V. Leis, P. Boncz, A. Kemper, Learned cardinalities: Estimating correlated joins with deep learning, in: CIDR, 2019.
- [15] M. Stillger, G. Lohman, V. Markl, M. Kandil, LEO – DB2’s learning optimizer, in: Proceedings of the 27th VLDB Conference, 2001, pp. 19–28.
- [16] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, N. Tatbul, Neo: A learned query optimizer, in: Proceedings of the VLDB Endowment, Vol. 12, 2019, pp. 1705–1718.
- [17] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, T. Kraska, Bao: Making learned query optimization practical, in: Proceedings of SIGMOD, 2022, pp. 1275–1288.